

## Day 17: Abstraction

- Abstract classes
- Interfaces

Today manam **Abstraction**, **Abstract Classes**, and **Interfaces** ni **zero knowledge nunchi deep ga** nerchukundam.

Nuvvu ee explanation ayyaka separate ga search cheyyalsina avasaram lekunda, **what, why, how, when, difference, syntax, examples, interview questions, common mistakes** anni cover chestha.

---

### 1. First: Abstraction ante enti?

Very simple ga cheppali ante:

**Abstraction = unnecessary implementation details ni hide chesi, user ki necessary information/functionality matrame expose cheyyadam.**

Real-life example chuddam.

#### **Example: ATM**

ATM lo nuvvu:

- Card insert chesthav
- PIN enter chesthav
- Withdraw click chesthav
- Amount enter chesthav

ATM lopala:

- Bank server ki request ela vellindi?
- Account balance ela verify chestundi?
- Cash machine ela communicate chestundi?
- Transaction database lo ela save chestundi?

ivi neku kanipinchavu.

Nuvvu **what to do** matrame telusukuntav.

**How it happens internally** teliyalsina avasaram ledi.

This is **Abstraction**.

WHAT

HOW

Withdraw money → internally ela process chestundo

Check balance → database/server ela work chestundo

Transfer money → security ela verify chestundo

User ki **WHAT** important.

Internal **HOW** hide chestham.

---

## 2. Java lo Abstraction enduku?

Suppose manaki:

Car

ane class undi.

Car ni drive cheyyadaniki user ki:

start()

accelerate()

brake()

ila operations kavali.

But internally:

fuel injection

engine combustion

gear mechanism

ECU

sensors

ivi ela work chestunnayo user ki teliyalsina avasaram ledu.

So Java lo manam implementation details ni hide chesi required methods ni provide cheyyadaniki abstraction use chestham.

### Main advantages

#### 1. Complexity reduce chestundi

User ki unnecessary internal details kanipinchavu.

## 2. Security improve chestundi

Internal implementation direct ga expose cheyyakunda control cheyyachu.

## 3. Maintainability

Internal implementation change chesina, external code change cheyyalsina requirement takkuva.

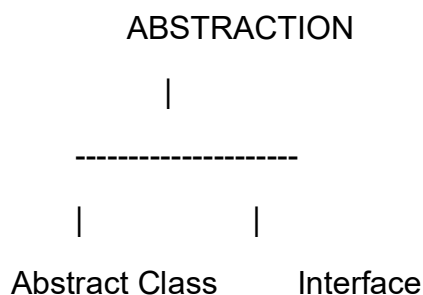
## 4. Loose coupling

Especially interfaces use chesinappudu components madhya dependency reduce cheyyachu.

---

## 3. Java lo Abstraction achieve cheyyadaniki two main ways

Java lo mainly:



So today manaki two major topics:

1. **Abstract Class**
2. **Interface**

---

## 4. Abstract Class ante enti?

First normal class gurinchi gurthu techukundam.

```
class Animal {
```

```
}
```

Idi normal class.

Dinni object create cheyyachu:

```
Animal a = new Animal();
```

But abstract class different.

```
abstract class Animal {
```

```
}
```

Ippudu:

```
Animal a = new Animal();
```

**✗ Not allowed**

Because Animal abstract class.

---

## 5. Why abstract class object create cheyyakudadhu?

Think logically.

Suppose:

```
abstract class Animal {
```

```
    abstract void sound();
```

```
}
```

Animal cheptundi:

"Every animal should have sound()."

But Animal itself exact ga em sound chestundo cheppaledu.

Dog:

Bark

Cat:

Meow

Cow:

Moo

So generic Animal ni direct ga create cheyyadam meaningful kaadu.

Instead:

```
Dog d = new Dog();
```

```
Cat c = new Cat();
```

---

## 6. Abstract method ante enti?

Important concept.

An abstract method is a method:

**Declaration untundi, implementation undadu.**

Example:

```
abstract void sound();
```

Observe:

```
abstract void sound();
```

↑

method

;

No body.

Normally method:

```
void sound() {  
    System.out.println("Sound");  
}
```

Here body exists.

But abstract method:

```
abstract void sound();
```

Body **undadu**.

---

## 7. Why implementation lekunda method?

Because parent class exact implementation decide cheyyakunda, child classes ki responsibility ivvadaniki.

Example:

```
abstract class Animal {
```

```
    abstract void sound();
```

```
}
```

Meaning:

"Every Animal should have a sound method, but exactly what sound is produced should be decided by the child."

Then:

```
class Dog extends Animal {
```

```
    void sound() {
```

```
        System.out.println("Bark");
```

```
    }
```

```
}
```

And:

```
class Cat extends Animal {
```

```
    void sound() {
```

```
        System.out.println("Meow");
```

```
    }
```

```
}
```

So:

Animal

```
|
```

```
|---- sound() → must exist
```

```
|
```

```
|—— Dog → Bark
```

```
|
```

```
└── Cat → Meow
```

This is abstraction.

---

## 8. Complete Abstract Class Example

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

Here two methods unnayi:

```
abstract void sound();  
  
and  
  
void eat() {  
    System.out.println("Animal eats");  
}
```

First one:

**abstract method**

Second one:

**concrete method**

---

## **9. Abstract class lo abstract + normal methods both undachu**

This is extremely important.

Abstract class:

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

}

Can contain:

Abstract methods ☒

Concrete methods ☒

Variables ☒

Constructors ☒

Static methods ☒

Final methods ☒

So abstract class is NOT a class where everything must be abstract.

---

## 10. Child class abstract method ni implement cheyyali

Suppose:

```
abstract class Animal {
```

```
    abstract void sound();
```

```
}
```

Now:

```
class Dog extends Animal {
```

```
}
```

✗ Error.

Why?

Because Dog inherited an abstract method:

sound()

but implementation provide cheyyaledu.

So:

```
class Dog extends Animal {
```



```
void sound() {  
    System.out.println("Bark");  
}  
}
```

Now correct.

---

## 11. @Override ikkada important

Manam previous day overriding nerchukunnam.

```
class Dog extends Animal {
```

```
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

Here Dog parent class lo unna abstract method ni implement chestundi.

@Override compiler ki:

"Nenu parent method ni override chestunnamu."

ani indicate chestundi.

### Important:

@Override method ni override cheyyadu.

Override already code behavior valla jarugutundi.

@Override just compiler ki check/confirmation.

---

## 12. Abstract class complete example

```
abstract class Animal {
```

```
    String name;
```

```
Animal(String name) {  
    this.name = name;  
}  
  
abstract void sound();  
  
void eat() {  
    System.out.println(name + " is eating");  
}  
}  
  
class Dog extends Animal {  
  
    Dog(String name) {  
        super(name);  
    }  
  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Main {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog("Tommy");  
    }  
}
```

```
        d.sound();  
        d.eat();  
    }  
}
```

Output:

Dog barks

Tommy is eating

---

### 13. Abstract class constructor untunda?

**Yes.**

Idi beginners ki confusing.

Manam abstract class object create cheyyalem:

Animal a = new Animal(); // ✗

But abstract class constructor **untundi**.

Example:

```
abstract class Animal {  
  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}
```

Child object create chesinappudu:

Dog d = new Dog();

First parent constructor execute avtundi.

Dog object creation

↓

Animal constructor

↓

Dog constructor

Because child object creation lo parent part kuda initialize avvali.

---

#### 14. Abstract class lo variables untaya?

Yes.

```
abstract class Animal {
```

```
    String name;
```

```
    int age;
```

```
}
```

Child class access cheyyachu.

```
class Dog extends Animal {
```

```
    void display() {
```

```
        System.out.println(name);
```

```
        System.out.println(age);
```

```
    }
```

```
}
```

So abstract class is still a **class**.

It can have state and behavior.

---

#### 15. Abstract class lo main() method undacha?

Yes.

Abstract class lo normal static method kuda undachu.

Even:

```
public static void main(String[] args)
```

```
undachu.
```

But generally abstract class ni main application entry point ga use cheyyadam uncommon.

---

## 16. Can abstract class have no abstract methods?

**Yes.**

Example:

```
abstract class Animal {  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

No abstract method.

Still valid.

Why abstract declare chestham?

Because manaki aa class direct objects create cheyyakudadani design requirement undachu.

---

## 17. Abstract class ni final cheyyacha?

**✗ No.**

Why?

final class means:

"This class cannot be inherited."

Abstract class generally intended to be inherited.

So:

abstract final class Animal

**✗ Invalid.**

Conceptually contradictory.

abstract → inherit me

final → don't inherit me

---

## 18. Abstract method private avvacha?

✗ No.

Why?

Private method child class ki visible kaadu.

But abstract method implementation child class provide cheyyali.

So:

```
private abstract void sound();
```

✗ Invalid.

---

### 19. Abstract method final avvacha?

✗ No.

final method override cheyyakudadhu.

Abstract method ki child implementation compulsory.

So:

```
abstract final void sound();
```

✗ Invalid.

Again contradictory.

---

### 20. Abstract class inheritance

Abstract class ni normal class laga:

extends

use chesi inherit chestham.

```
abstract class Animal {  
    abstract void sound();  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

```
}  
}
```

---

## 21. Abstract class can extend another abstract class

Yes.

```
abstract class Animal {  
  
    abstract void sound();  
}
```

Then:

```
abstract class Dog extends Animal {  
  
}
```

Dog itself abstract kabatti sound() implement cheyyalsina compulsory ledi.

Then:

```
class Puppy extends Dog {  
  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

So:

Animal

↓

abstract Dog

↓

Puppy

---

## 22. Now Interfaces

Now second major topic.

### Interface ante enti?

Simple definition:

**Interface is a contract that specifies what a class must do, without primarily defining how it does it.**

Example:

```
interface Animal {  
  
    void sound();  
}
```

This says:

"Any class that implements Animal must provide sound()."

---

## 23. Interface ni real-life example tho understand cheddham

Suppose college lo rule:

"Every student must submit an assignment."

College exact ga:

"Assignment ela rayali?"

ani completely decide cheyyakapovachu.

Student:

Student A → handwritten

Student B → typed

Student C → printed

But requirement same:

submitAssignment()

This is similar to interface.

Interface:

```
interface Student {
```



```
void submitAssignment();  
}
```

Different classes different implementation ivvachu.

---

## 24. Interface syntax

```
interface Animal {
```

```
    void sound();  
}
```

Then class:

```
class Dog implements Animal {
```

```
    @Override  
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

Notice:

Abstract class:

extends

Interface:

implements

---

## 25. implements meaning

When we write:

```
class Dog implements Animal
```

Meaning:

Dog class promises to follow the Animal interface contract.

Interface says:

```
void sound();
```

Dog must provide it.

---

## **26. Interface methods are public**

Important point.

An interface method declared like:

```
interface Animal {
```

```
    void sound();
```

```
}
```

is implicitly:

```
public abstract void sound();
```

So this:

```
void sound();
```

means essentially:

```
public abstract void sound();
```

Therefore implementation:

```
class Dog implements Animal {
```

```
    public void sound() {
```

```
        System.out.println("Bark");
```

```
    }
```

```
}
```

must be public.

---

## **27. Why public?**

Interface contract ni outside classes implement cheyyali.

If implementation visibility is reduced:

void sound()

instead of:

public void sound()

then compiler error vastundi.

Because parent/interface method is public.

Child implementation cannot reduce visibility.

---

## 28. Interface variables

Interface lo variables by default:

public

static

final

So:

interface Animal {

int LEGS = 4;

}

actually means:

public static final int LEGS = 4;

So value change cheyyalem.

Animal.LEGS = 5;

✗ Error.

Because final.

---

## 29. Interface variables are constants

Usually naming convention:

MAX\_SPEED

PI

MIN\_VALUE

Example:

```
interface Vehicle {
```

```
    int MAX_SPEED = 120;
```

```
}
```

Access:

```
System.out.println(Vehicle.MAX_SPEED);
```

---

### 30. Can interface have constructor?

✗ No.

Why?

Interface object direct ga create cheyyalem.

```
Animal a = new Animal();
```

✗

Constructor purpose is object initialization.

Interface itself object create cheyyadu.

---

### 31. Can we create interface object?

No.

```
Animal a = new Animal();
```

✗

But very important:

```
Animal a = new Dog();
```

✓

This is allowed.

Why?

Dog is actual object.

Animal is reference type.

---

### 32. This is extremely important

Consider:

```
interface Animal {
```

```
    void sound();
```

```
}
```

```
class Dog implements Animal {
```

```
    public void sound() {
```

```
        System.out.println("Bark");
```

```
    }
```

```
}
```

Now:

```
Animal a = new Dog();
```

Here:

Animal → reference type

Dog → actual object

This is **polymorphism**.

Then:

```
a.sound();
```

Output:

Bark

Because actual object is Dog.

---

### 33. Why use interface reference?

Suppose:

```
interface Payment {
```

```
    void pay();
```

```
}
```

Implementations:

```
class UPI implements Payment {
```

```
    public void pay() {
```

```
        System.out.println("UPI payment");
```

```
    }
```

```
}
```

```
class Card implements Payment {
```

```
    public void pay() {
```

```
        System.out.println("Card payment");
```

```
    }
```

```
}
```

Now:

```
Payment p;
```

We can assign:

```
p = new UPI();
```

```
p.pay();
```

or:

```
p = new Card();
```

```
p.pay();
```

Same interface reference.

Different implementations.

This gives **flexibility**.

---

### 34. Interface supports multiple inheritance

This is one of the biggest reasons interfaces are important.

Java does NOT allow multiple inheritance through classes.

This is invalid:

```
class C extends A, B
```

✗

Why?

Because ambiguity problems can occur.

But Java allows:

```
class C implements A, B
```

✓

Example:

```
interface Camera {  
    void takePhoto();  
}
```

```
interface MusicPlayer {  
    void playMusic();  
}
```

```
class Smartphone implements Camera, MusicPlayer {
```

```
    public void takePhoto() {  
        System.out.println("Taking photo");  
    }
```

```
    public void playMusic() {  
        System.out.println("Playing music");  
    }  
}
```

Now:

Camera

\

\

Smartphone

/

/

MediaPlayer

One class multiple interfaces implement cheyyachu.

---

### 35. Abstract class vs Interface

Very important interview topic.

| Abstract Class                                            | Interface                                                        |
|-----------------------------------------------------------|------------------------------------------------------------------|
| abstract class                                            | interface                                                        |
| Class ni extends chestham                                 | Interface ni implements chestham                                 |
| One class only extend cheyyagaladu                        | Multiple interfaces implement cheyyachu                          |
| Variables normal ga undachu                               | Variables implicitly public static final                         |
| Constructors allowed                                      | Constructors not allowed                                         |
| Abstract + concrete methods                               | Abstract methods + modern Java lo default/static/private methods |
| Instance variables possible                               | Instance state variables generally not possible                  |
| Can have any access modifiers on members subject to rules | Interface abstract methods are public                            |
| Represents shared base/identity + behavior                | Represents contract/capability                                   |

---

### 36. Abstract class when to use?

Suppose:

Animal

Dog



Cat

Cow

All animals have common properties:

name

age

eat()

sleep()

But sound differs.

So:

```
abstract class Animal {
```

```
    String name;
```

```
    int age;
```

```
    void eat() {
```

```
        System.out.println("Eating");
```

```
    }
```

```
    abstract void sound();
```

```
}
```

This is a good use case.

Because we have:

**common state + common behavior + some specialized behavior**

---

### 37. Interface when to use?

Suppose:

Flyable

Swimmable

Playable

Payable

Printable

These are capabilities.

Example:

```
interface Flyable {  
    void fly();  
}
```

Different unrelated classes can implement it.

```
class Bird implements Flyable {  
    public void fly() {  
        System.out.println("Bird flies");  
    }  
}  
  
class Airplane implements Flyable {  
    public void fly() {  
        System.out.println("Airplane flies");  
    }  
}
```

Bird and Airplane are not the same type of thing, but both have the capability:

fly()

That's a strong interface use case.

---

### 38. Easy way to remember

**Abstract class = "IS-A base"**

Dog IS-A Animal

Cat IS-A Animal

So:

class Dog extends Animal

**Interface = "CAN-DO capability"**

Bird CAN fly

Airplane CAN fly

So:

class Bird implements Flyable

class Airplane implements Flyable

Remember:

**Abstract class** → **common identity/base**

**Interface** → **common capability/contract**

---

### 39. Can a class extend abstract class and implement interface?

Absolutely.

```
abstract class Animal {
```

```
    abstract void sound();
```

```
}
```

```
interface Flyable {
```

```
    void fly();
```

```
}
```

```
class Bird extends Animal implements Flyable {
```

```
    public void sound() {
```

```
        System.out.println("Chirp");
```

```
    }
```

```
    public void fly() {
```

```
        System.out.println("Bird flies");
```

```
}  
}
```

Here:

Bird

```
|  
|—— extends Animal  
  
|  
|—— implements Flyable
```

---

#### 40. Can abstract class implement interface?

Yes.

```
interface Animal {
```

```
    void sound();  
}
```

Then:

```
abstract class Dog implements Animal {
```

```
}
```

Because Dog is abstract, it doesn't necessarily need to implement sound() yet.

Later:

```
class Puppy extends Dog {
```

```
    public void sound() {  
        System.out.println("Bark");  
    }  
}
```

---

#### 41. Can interface extend another interface?

Yes.

But keyword is:

extends

Example:

```
interface Animal {
```

```
    void eat();
```

```
}
```

```
interface Dog extends Animal {
```

```
    void bark();
```

```
}
```

Then:

```
class Puppy implements Dog {
```

```
    public void eat() {
```

```
        System.out.println("Eating");
```

```
    }
```

```
    public void bark() {
```

```
        System.out.println("Barking");
```

```
    }
```

```
}
```

---

## 42. Interface can extend multiple interfaces

Yes.

```
interface A {
```

```
    void methodA();
```

```
}
```

```
interface B {  
    void methodB();  
}
```

```
interface C extends A, B {  
    void methodC();  
}
```

Then:

```
class Demo implements C {  
  
    public void methodA() {  
    }  
  
    public void methodB() {  
    }  
  
    public void methodC() {  
    }  
}
```

This is another way Java supports multiple inheritance of type/contracts.

---

## **43. Modern Java interfaces**

Earlier Java interfaces were mainly:

abstract methods

constants

But modern Java allows more.

Interfaces can contain:

### **1. Abstract methods**

```
void sound();
```

## 2. Default methods

```
default void eat() {  
    System.out.println("Eating");  
}
```

## 3. Static methods

```
static void info() {  
    System.out.println("Animal interface");  
}
```

## 4. Private methods

Modern Java interfaces can also contain private helper methods.

But for your current level, most important are:

abstract

default

static

---

## 44. Default method

Suppose:

```
interface Animal {
```

```
    void sound();
```

```
    default void eat() {  
        System.out.println("Animal eats");  
    }
```

```
}
```

A class implements Animal:

```
class Dog implements Animal {
```

```
public void sound() {  
    System.out.println("Bark");  
}  
}
```

Dog doesn't implement eat().

Still:

```
Dog d = new Dog();
```

```
d.eat();
```

works.

Because interface already provided default implementation.

---

#### **45. Why default methods were introduced?**

Imagine an interface already used by 100 classes:

```
interface Payment {
```

```
    void pay();  
}
```

Now interface creator wants to add:

```
void refund();
```

All 100 classes would need to implement it.

That could break existing code.

Default method allows:

```
default void refund() {  
    System.out.println("Refund");  
}
```

Existing classes can continue working.

---

#### **46. Static method in interface**



Example:

```
interface Utility {  
  
    static void display() {  
        System.out.println("Hello");  
    }  
}
```

Call using interface name:

```
Utility.display();
```

Not:

```
new Dog().display();
```

Interface static methods belong to the interface itself.

---

## **47. Abstract class vs Interface — deeper understanding**

Suppose you have:

```
abstract class Vehicle
```

You may store:

speed

brand

price

engine

because these are properties/state.

And:

```
start()
```

```
stop()
```

common behavior.

Then specific behavior:

```
abstract void move();
```

This makes sense.

But:

interface Flyable

doesn't need to store:

wingSize

wingWeight

featherCount

Its main purpose is:

fly()

contract.

So:

Abstract class → "What this thing fundamentally is"

Interface → "What this thing can do"

---

## 48. Abstraction vs Encapsulation

Since yesterday we studied Encapsulation, this difference is VERY important.

### Encapsulation

Data and methods ni one unit lo bundle chesi, data ni protect/hide cheyyadam.

Example:

```
class BankAccount {
```

```
    private double balance;
```

```
    public double getBalance() {
```

```
        return balance;
```

```
    }
```

```
}
```

Here:

private → data protection

getter → controlled access

## Abstraction

Implementation complexity ni hide chesi necessary functionality ni expose cheyyadam.

Example:

```
abstract class Vehicle {  
  
    abstract void start();  
}
```

User ki:

start()

telusu.

Internal engine mechanism hidden.

---

## 49. Easy difference

Remember this:

ENCAPSULATION

"What data can you access?"

ABSTRACTION

"What details do you need to know?"

Or:

Encapsulation → HOW to protect data

Abstraction → HOW to hide complexity

---

## 50. Abstraction + Polymorphism together

Real Java applications lo these concepts often work together.

Example:

```
interface Payment {
```

```
    void pay();  
}
```

Different implementations:

```
class UPI implements Payment {
```

```
    public void pay() {  
        System.out.println("UPI payment");  
    }  
}
```

```
class Card implements Payment {
```

```
    public void pay() {  
        System.out.println("Card payment");  
    }  
}
```

Now:

```
Payment p = new UPI();
```

```
p.pay();
```

Output:

UPI payment

Then:

```
p = new Card();
```

```
p.pay();
```

Output:

Card payment

Here:

### **Abstraction**

User works with:

Payment

instead of knowing implementation details.

## Polymorphism

Same:

p.pay();

different behavior depending on object.

---

### 51. Very important mental model

Suppose food delivery application ki payment process design chestunnam.

We don't want application code everywhere to know:

UPI API

Card API

Bank API

Wallet API

Instead:

interface Payment {

void pay();

}

Then:

Payment

|

├── UPI

├── Card

└── Wallet

Application only says:

payment.pay();

Which payment mechanism actually runs depends on implementation.

This is the power of abstraction.

---

## 52. Why interfaces are heavily used in real projects?

Because they help us design flexible systems.

For example:

```
interface NotificationService {  
  
    void send(String message);  
}
```

Implementations:

EmailNotification

SMSNotification

WhatsAppNotification

PushNotification

Application doesn't need to depend heavily on a specific implementation.

It can depend on:

NotificationService

This is useful for:

- testing
- replacing implementations
- scaling applications
- loose coupling
- clean architecture

---

## 53. One complete example combining everything

```
abstract class Vehicle {
```

```
    String brand;
```

```
    Vehicle(String brand) {
```

```
        this.brand = brand;
```

```
}
```

```
abstract void start();
```

```
void stop() {
```

```
    System.out.println("Vehicle stopped");
```

```
}
```

```
}
```

Interface:

```
interface Electric {
```

```
    void charge();
```

```
}
```

Class:

```
class Tesla extends Vehicle implements Electric {
```

```
    Tesla(String brand) {
```

```
        super(brand);
```

```
}
```

```
@Override
```

```
void start() {
```

```
    System.out.println("Electric vehicle started");
```

```
}
```

```
@Override
```

```
public void charge() {
```

```
    System.out.println("Charging battery");
```

```
}
```

```
}
```

Main:

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Tesla t = new Tesla("Tesla");
```

```
        t.start();
```

```
        t.charge();
```

```
        t.stop();
```

```
    }
```

```
}
```

Output:

Electric vehicle started

Charging battery

Vehicle stopped

Here:

Vehicle

↓

Abstract class

↓

common state + common behavior + abstract behavior

Electric

↓

Interface

↓

capability/contract



Tesla



extends Vehicle

implements Electric

---

#### 54. One more important concept: abstract class reference

Just like interface reference:

```
abstract class Animal {
```

```
    abstract void sound();
```

```
}
```

```
class Dog extends Animal {
```

```
    void sound() {
```

```
        System.out.println("Bark");
```

```
    }
```

```
}
```

We can do:

```
Animal a = new Dog();
```

Can we?

**Yes.**

But:

```
Animal a = new Animal();
```

**✗ No.**

Again:

Reference type → Animal

Actual object → Dog

Then:

```
a.sound();
```

Dog's implementation executes.

This is runtime polymorphism + abstraction.

---

## 55. Common beginner confusion

**"Abstract class object create cheyyalem, then reference enduku?"**

Because reference and object are different.

```
Animal a = new Dog();
```

Animal is only reference type.

Actual object:

Dog

So abstract class can be used as a **reference type**.

---

## 56. Can abstract class have private methods?

Yes.

Example:

```
abstract class Animal {  
  
    private void helper() {  
        System.out.println("Helper");  
    }  
  
    abstract void sound();  
}
```

A private concrete method is allowed.

But:

```
private abstract void sound();
```

✗ Not allowed.

Because abstract method needs overriding/implementation by subclass.

---

### 57. Can abstract class have static methods?

Yes.

```
abstract class Animal {  
  
    static void info() {  
        System.out.println("Animals");  
    }  
}
```

Call:

```
Animal.info();
```

---

### 58. Can abstract class have final methods?

Yes.

```
abstract class Animal {  
  
    final void breathe() {  
        System.out.println("Breathing");  
    }  
}
```

Child cannot override:

```
@Override
```

```
void breathe()
```

✗

Because final.

Notice the difference:

abstract class + final method → possible

abstract method + final → impossible

---

## **59. Interface vs Abstract Class — interview-level decision**

Suppose question:

"When should I choose abstract class?"

Think:

### **Choose abstract class when:**

- classes have strong parent-child relationship
- common state is needed
- constructors are needed
- common implementation is needed
- you want to provide both abstract and concrete behavior

Example:

Animal → Dog, Cat

Vehicle → Car, Bike

Employee → Manager, Developer

---

### **Choose interface when:**

- you want a contract
- unrelated classes should share a capability
- multiple inheritance of type is required
- you want loose coupling
- implementation can vary independently

Example:

Flyable

Payable

Printable

Serializable

Runnable

## 60. The biggest difference in one example

Suppose:

Bird

Airplane

Both can fly.

But:

Bird IS-A Animal

Airplane IS-NOT-A Animal

So:

```
interface Flyable {  
    void fly();  
}
```

Both can:

class Bird implements Flyable

class Airplane implements Flyable

This is perfect.

But:

abstract class Animal

only makes sense for things that are actually animals.

---

## 61. Abstract Class vs Interface — final revision table

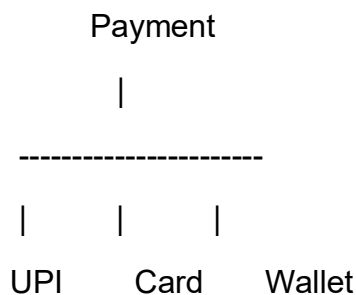
| Feature            | Abstract Class | Interface |
|--------------------|----------------|-----------|
| Keyword            | abstract class | interface |
| Object creation    | ✗              | ✗         |
| Reference possible | ✓              | ✓         |

| Feature               | Abstract Class                       | Interface                                                      |
|-----------------------|--------------------------------------|----------------------------------------------------------------|
| Inheritance keyword   | extends                              | implements                                                     |
| Interface inheritance | —                                    | extends                                                        |
| Multiple inheritance  | ✗ through classes                    | ✓ multiple interfaces                                          |
| Constructor           | ✓                                    | ✗                                                              |
| Instance variables    | ✓                                    | ✗ as normal mutable instance fields                            |
| Constants             | ✓ possible                           | ✓ implicitly public static final                               |
| Abstract methods      | ✓                                    | ✓                                                              |
| Concrete methods      | ✓                                    | ✓ via default/static/private mechanisms                        |
| Static methods        | ✓                                    | ✓                                                              |
| Final methods         | ✓                                    | Interface rules differ; abstract contract methods aren't final |
| Main purpose          | Shared base + partial implementation | Contract/capability                                            |

## 62. Don't memorize abstraction blindly

Instead remember this story.

Imagine:



Payment says:

"Every payment mechanism must provide pay()."

But it doesn't force application to know:

UPI API ela call chestundi?

Card verification ela jarugutundi?

Wallet server ela communicate chestundi?

That's abstraction.

Then:

```
Payment p = new UPI();
```

```
p.pay();
```

The application knows:

WHAT → pay()

It doesn't need to know:

HOW → UPI internally works

---

## **63. One-line definitions for exams**

### **Abstraction**

**Abstraction is the process of hiding implementation details and exposing only essential functionality to the user.**

### **Abstract Class**

**An abstract class is a class declared with the abstract keyword that cannot be instantiated and can contain both abstract and concrete methods.**

### **Abstract Method**

**An abstract method is a method declared without a body, whose implementation is provided by a concrete subclass.**

### **Interface**

**An interface is a contract that defines a set of operations that implementing classes must provide.**

### **implements**

**The implements keyword is used by a class to implement an interface.**

---

## 64. Most important syntax patterns

### Abstract class

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

### Child class

```
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

### Interface

```
interface Flyable {  
  
    void fly();  
}
```

### Implementation

```
class Bird implements Flyable {  
  
    @Override  
    public void fly() {  
        System.out.println("Flying");  
    }  
}
```



```
}  
}
```

### Multiple interfaces

```
class Smartphone implements Camera, MediaPlayer {  
  
}
```

### Abstract class + interface

```
class Bird extends Animal implements Flyable {  
  
}
```

---

## 65. Common mistakes you should never make

### Mistake 1

```
Animal a = new Animal();
```

✗ If Animal is abstract.

---

### Mistake 2

```
interface Animal {
```

```
    Animal() {  
    }  
}
```

✗ Interface cannot have constructor.

---

### Mistake 3

```
class Dog implements Animal {
```

```
    void sound() {  
    }
```

```
}
```

If interface method is public:

✗ Access is weaker.

Correct:

```
public void sound() {  
}
```

---

#### **Mistake 4**

```
abstract final void sound();
```

✗ Abstract + final method impossible.

---

#### **Mistake 5**

```
private abstract void sound();
```

✗ Private abstract method impossible.

---

#### **Mistake 6**

Thinking:

"Abstract class means all methods are abstract."

✗ Wrong.

It can have both:

abstract methods

+

concrete methods

---

#### **Mistake 7**

Thinking:

"Interface can never have method body."

That was the traditional model, but modern Java interfaces can have default, static, and private methods with bodies.

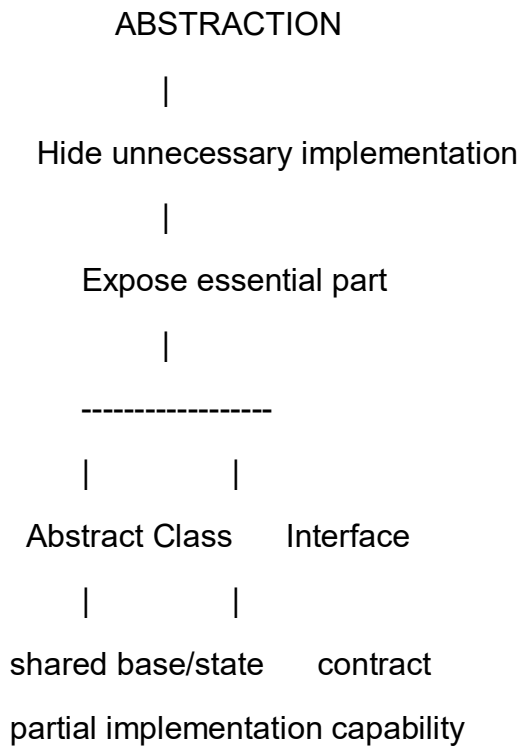
---

## 66. Your complete mental picture

If I ask:

**What is abstraction?**

Think:



Then:

Abstract class

↓

extends

↓

Child class

and:

Interface

↓

implements

↓

Class

And:

One class

|

|— extends ONE class

|

└— implements MANY interfaces

This last line is **very important**.

---

## 67. Final comparison with everything you've learned so far

You have now learned several OOP concepts.

Think of them together:

ENCAPSULATION

↓

Protect data

↓

private + getters/setters

INHERITANCE

↓

Reuse/derive from existing class

↓

extends

POLYMORPHISM

↓

Same interface/method call

different behavior



overloading / overriding

## ABSTRACTION



Hide implementation complexity



abstract class / interface

So these concepts are **not isolated topics**.

They work together to build proper object-oriented applications.

---

### 68. The most important 10 things to remember from Day 17

If tomorrow exam/interview mundu only 10 points revise cheyyali ante:

1. **Abstraction = hide implementation details, expose essential functionality.**
2. Java lo abstraction mainly **abstract classes and interfaces** dwara achieve chestham.
3. **Abstract class object direct ga create cheyyalem.**
4. Abstract class can contain **abstract + concrete methods**.
5. Abstract method has **no method body**.
6. Concrete child class must implement inherited abstract methods.
7. Interface ni class **implements** chestundi.
8. Interface itself direct ga object create cheyyalem.
9. One class **one class ni extend** cheyyagaladu, but **multiple interfaces implement** cheyyagaladu.
10. Easy memory trick:

Abstract Class → IS-A / common base

Interface → CAN-DO / contract

**Final example to keep in your mind**

```

abstract class Animal {

    abstract void sound();

    void eat() {
        System.out.println("Eating");
    }
}

interface Flyable {

    void fly();
}

class Bird extends Animal implements Flyable {

    @Override
    void sound() {
        System.out.println("Chirp");
    }

    @Override
    public void fly() {
        System.out.println("Flying");
    }
}

```

Read this as English:

**Bird IS-A Animal, so it extends Animal.**

**Bird CAN fly, so it implements Flyable.**

**Animal says every animal must have sound(), but doesn't decide the exact**

**sound.**

**Bird provides the actual sound.**

**Flyable says something must be able to fly, but Bird decides how it flies.**

**That's abstraction.**